

030 实验指南

1 要证明的工程承诺

本实验检验三件事：数据职责在代码里没有混淆；每批损失正确汇总；完整epoch边界中断后，下一段训练确实延续同一状态轨迹。一次成功导入torch，或者两次最后loss看起来接近，都不足以完成这些目标。

先读正文第2—7节。输入是data中的72行合成数据、固定拆分与配置，不含现实个人资料。网络 $2 \rightarrow 4 \rightarrow 1$ ，17个参数，训练使用tanh与Dropout(0.25)。训练45行、验证15行、测试12行；12轮、每批最多7行，合计84次参数更新。没有GPU、模型下载或付费API。

2 环境与第一次运行

在本目录下使用独立Python环境，版本和安装说明见README与environment.yml。实际验证为Linux CPU、Python3.12.14、PyTorch2.7.1+cpu、NumPy2.3.5；全新Anaconda安装、浏览器Jupyter、GPU及跨平台未实测。

```
python experiment.py
python -m unittest -v test_experiment
```

脚本先校验三份输入原始字节，再完成所有计算和序列化，最后写入outputs。可指定一个新目录，便于与提供的结果对照：

```
python experiment.py --output my-run
```

不要手工修改data后继续把图与原结论当成相同实验。固定报告入口拒绝变化后的输入；探索参数时，在新的Python文件中调用明确的函数并另存结果。也不要加载不可信的.pt文件；提供的检查点只含本课程自己生成的Tensor与基础容器。

3 先审计模型和批次

运行以下完整代码，它只构造对象，不更新参数。打印的四组参数共有17个标量，另有mean与scale两个buffer。

```
import experiment as e
x, y, split, cfg = e.load_inputs()
run = e.make_run(x, y, split, cfg)
model = run['model']
print([(n, tuple(p.shape)) for n, p in model.named_parameters()])
print([(n, tuple(b.shape)) for n, b in model.named_buffers()])
rows = list(run['loaders']['train'])
print([len(batch[1]) for batch in rows])
print(sorted(i for batch in rows for i in batch[2].tolist()))
```

最后两行分别应显示六个7与一个3，以及45个训练ID。注意：迭代训练loader会推进它自己的随机状态。若想从正式初始化开始训练，应重新调用make_run，而不是在调试时提前遍历之后还声称保留原始轨迹。构造时复制数据、取样时返回副本的行为，也可通过test_01检查。

4 手算一次更新与不等长归约

先不用小网络，完成正文线性模型的两行例子。写出每个残差、损失对预测的导数、w与b梯度，再与答案核对；不要只抄最后的1.3与0.2。

接着读正文5.4节，并打开outputs/first-step-trace.json。沿ID56追踪x、标准化、仿射、tanh、掩码、预测、残差及其反向各量；再将七行贡献相加得到共享参数梯度。核对全部17个更新，最后用同一数据做固定掩码与eval两种前向，解释为什么不能混用模式比较。test_12对这一份轨迹独立验证。

随后用两个批次的手工数据核查统计：2行均值1、3行均值4，整体应为2.8，不是2.5。打开outputs/history.csv，第4轮正确验证MSE约0.10013470，错误的批次等权值约0.16671112。解释最后1行批次为什么被错误规则放大。

损失用于本批梯度时仍采用本批均值；这里的加权是在计算跨批报告指标。连续多次更新之间参数改变，所以不能把逐批训练误差汇总解释为“同一个固定模型的全训练集风险”。正文图7与第6节专门区分这两件事。

5 审计验证函数

阅读evaluate。列出进入前保存了什么、作用域内允许什么、退出后检查什么。代码同时使用eval和no_grad，并恢复各子模块原先的模式。test_04故意制造混合模式，验证恢复没有把它们粗暴地统一成根模块的一个布尔值。

观察outputs/semantics.json。两个故障结论都应true：eval仍能执行反向和step；train模式配no_grad时Dropout仍能给出不同输出。这里没有把no_grad说成会修改已有Tensor的requires_grad标志。

如果要故意在验证中更新权重，请在一个新建模型副本上写故障探针，不要修改正式evaluate后继续复用已生成的报告。故障报告要指明究竟改变了参数、buffer、grad、模式还是随机状态，不要仅写“验证不稳定”。

6 真正关闭进程后恢复

正常运行会产生outputs/checkpoint-epoch-04.pt。它记录第4轮全部训练与验证完成后的状态，包括当前权重、动量、两个随机状态、配置、拆分和最佳候选。

关闭当前Python进程，另开终端启动。以下反斜杠续行适用于Bash/zsh；其他终端可把三行接为一条命令并去掉反斜杠：

```
python experiment.py \
  --resume-checkpoint outputs/checkpoint-epoch-04.pt \
  --output resumed-run
```

新进程输出resumed-run/resume-final-state.json。用以下独立程序比较全部最终状态的序列化文本，而不是只比较loss：

```
from pathlib import Path
left = Path('outputs/final-state.json').read_bytes()
right = Path('resumed-run/resume-final-state.json').read_bytes()
if left != right:
    raise RuntimeError('最终完整状态不同，需要定位第一个差异')
print('同一环境中的完整最终状态逐字节一致')
```

构建验证确实执行了这一新进程路线。在Notebook里重新建立对象只证明对象恢复逻辑；它本身不能冒充“已经重启操作系统进程”。本讲两种路径都分别有执行证据。

7 删去一种状态观察偏差

run_report已经分别调用restore的omit参数，故意不恢复optimizer、torch_rng或loader_rng。只删除一种，以便定位原因。查看summary.json与正文图9：

- 缺动量：最终参数最大绝对差约0.03149809，样本顺序仍相同
- 缺全局RNG：差约0.24162864，样本顺序仍相同，但Dropout掩码不同
- 缺加载器RNG：差约0.33410590，后续样本顺序不同

完整恢复则比较全部checkpoint结构并得到相等。一次结果的偏差大小不能作为三类状态的普遍重要性排名，也不能把“不同轨迹也收敛到了不错的loss”当成精确恢复成功。

8 测试职责与失败保护

test_07改变测试特征和标签，验证训练与选择不变；再改变验证标签，验证梯度训练轨迹不变而验证日志变化。这是代码干预检查，不是重新利用测试数字挑选超参数。

test_10针对损坏的epoch、配置、拆分、模型shape或数值、动量、随机状态等构造17种拒绝案例。test_11修改输入字节，在不存在的输出目录和已有sentinel目录上分别执行，检查失败不会创建新报告或覆盖旧文件。这些检查不构成任意恶意文件安全性证明，也不提供多个输出文件在断电场景下的整体事务原子性。

独立Notebook从清空内核顺序执行。第一个代码单元先验证固定输入与提供的输出，之后在临时目录重新运算并逐文件比较。若原始输出已改，先恢复原始版本再运行，而不是跳过检查单元。

9 提交一份可定位的故障报告

提交四项证据：参数/buffer登记表；完整与错误损失归约；新进程完整恢复比较；三种缺失状态的隔离实验。每个故障写出预期不变量、实际差异、最早出现差异的位置、最小修复以及复测结果。

例如“第5轮样本顺序首次不同，参数在首批更新后不同；配置和模型读取相同，加载器RNG未载入；恢复generator状态后全部后续状态一致”比“训练无法复现”更有用。最后注明仅验证CPU单进程、固定环境与完整epoch边界；不要把该报告扩展成未运行的GPU、多worker或中间批次恢复结论。